Randomized Online-DRL vs Online-DRL

What improves, what it costs, and how every metric is computed

Evidence from drl-sim - the simulator of

Edge Service Caching With Delayed Hits (Deka & Sukapuram, IEEE TCC 2025) and of the

Randomized DRL with Multi-Resource (CPU, RAM, Disk) Capacity proposal

Synthetic Zipf/Poisson traffic + the paper's adversarial Large Sequence; the offline optimum is an exact MILP (HiGHS).

All numbers are read from results/.json - nothing is hand-written.*

The question

and the short answer

- How does each algorithm decide what to keep?
- Does the randomized algorithm IMPROVE the deterministic one, or does it only EQUALIZE it?
- Where does it LOSE?
- How are the metrics (ratio, cost, utilization, regret, running time) actually calculated?

Short answer:

it strictly improves the competitive ratio on the adversarial family the paper analyses (gamma = 2k -> ~3, flat in k), it EQUALIZES the deterministic algorithm on ordinary traffic (1.443 vs 1.430) while spending 3x less download cost, it loses on short traces (1.279 vs 1.145), on CPU time (25-70x more per request) and on the literal form of goal G3.

The common setting

the environment both algorithms run in

- An edge node caches N services; a service has a forwarding latency l, a download time M and a demand vector (CPU, RAM, disk).
- Latency and cost are kept separate: latency = what the user waits, cost = the download time the operator pays.

event	condition	latency	cost
Hit	the service is cached	0	-
Delayed hit	a download is in flight and the remaining time <= l (the request is buffered)	min(l, remaining)	the download already paid
Request forwarding	not cached, no download in flight (or the remaining time > l)	l	-

Why it matters:
every request is charged its latency, every started download its M; the two algorithms are measured with exactly the same engine, so every difference in the next slides is a difference in CACHE MANAGEMENT, not in the environment.

How Online-DRL works

the paper: deterministic, per-service state

- TRIGGER (Algorithm 1). At the first request of a miss epoch a download starts iff T_i >= theta*M_i or L_i >= theta*M_i, where T_i = time since the epoch began and L_i = l * (number of requests of the service in the epoch). theta = 1 in the paper, theta = 0 is LL-RC.
- EVICTION (Algorithm 2, Landlord). Every cached service carries a credit, initialised to M and reset on every hit. When a new service does not fit, the credits are decreased proportionally; the services whose credit hits 0 become eligible and are evicted in increasing order of the resources they use beyond the unmet need.
- ADMISSION. A service enters only if it fits in ALL THREE resources and the cardinality C.
- Guarantee: competitive ratio gamma in O(k(1 + 3 beta_up M_x)); the proof exhibits an adversarial family on which gamma = Theta(k).

How Randomized Online-DRL works

the proposal: R1 + R2 + R3

- R1 - a FRACTIONAL state + projected OGD. Instead of a credit per service the algorithm keeps p in D_vec = {p in [0,1]^N : sum p <= C, sum a_n,r p_n <= 1 for every r}. Each request takes one gradient step on the requested service and projects back onto D_vec with a dual-ascent projection (eqs. 22-25); eta = sqrt(N)/(l_max sqrt(T)).
- R2 - RANDOMIZED ROUNDING with repair. K persistent uniform vectors give the paths s_k = 1[U_k < p], so E[s] = p and E|ds| = |dp|. A path that violates the constraints is repaired by evicting services. The real cache = (downloaded) AND (sampled on): a service that was sampled but never downloaded is NOT a hit.
- R3 - a RANDOMIZED download threshold, drawn from f(theta) = e^(theta/M_n)/(M_n(e-1)) on [0, M_n]: the rent-or-buy ratio 2 becomes e/(e-1) = 1.582.
- BUDGET awareness. The trigger probability is lowered to budget_left/(M_n * requests_left) so that the expected download cost stays below B_max (goal G2).

Side by side

what actually differs

aspect	Online-DRL	Randomized Online-DRL
state	one credit per cached service	fractional p in D_vec + K rounding paths
download decision	deterministic counters vs a fixed theta	same counters, random threshold, budget-aware probability
eviction	Landlord: credit reaches 0, then a greedy resource-aware order	the OGD state lowers p, the rounding switches the service off, violating paths are repaired
entering the cache	when the download completes	when the download completes AND the serving path samples it on
guarantee	gamma in O(k(1+3 beta_up M_x))	surrogate regret <= sqrt(N) l_max sqrt(T); expected cost <= B_max
resource feasibility	checked exactly at admission	guaranteed by the repair step
price	no randomness, no learning	25-70x more CPU per request; needs a warm-up

How the metrics are calculated (1/2)

the objective quantities

metric	formula	why it is the right one
latency	sum_t lat_t, with lat_t = 0 (hit), min(l, remaining) (delayed hit) or l (forward)	exactly the objective of the paper
cost	sum of M_n over the STARTED downloads	the operator's cost; forwarding is charged through the latency, not the cost
downloads	number of started downloads	the download budget is spent per download
competitive ratio gamma	ALG(I)/OPT(I); OPT = MILP optimum (HiGHS) or the paper's (k+2n)l on the Large Sequence	the paper's definition; if the MILP hits its time limit the ratio is only a lower bound
fit of gamma(k)	least squares: linear / logarithmic / power law (log-log) with R^2	shows empirically whether the linear behaviour disappears - it is not a proof
beta_up, beta_down	per service fwd_i/M_i, where fwd_i counts the requests forwarded WHILE a download of i was in flight	the paper's beta is exactly that fraction; beta_up = max (with M_x), beta_down = mean (with M_min)

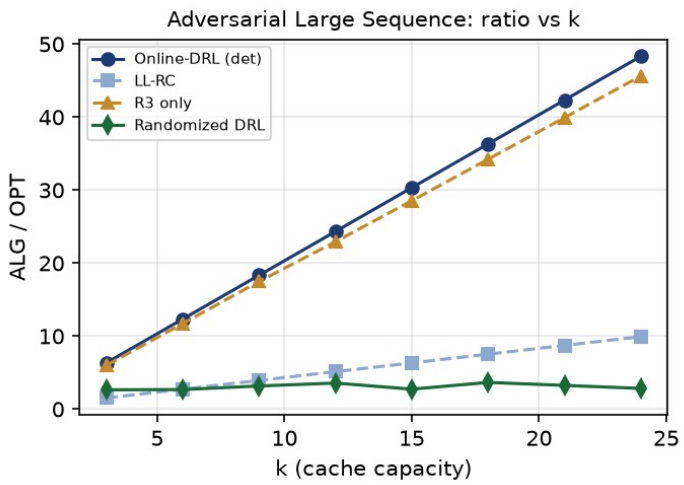
How the metrics are calculated (2/2)

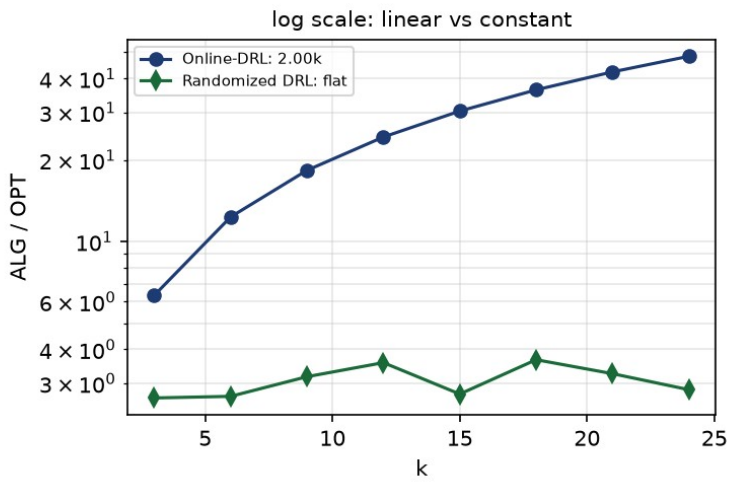
the resource, regret and cost metrics

metric	formula	why it is the right one
U_r,t, U^max	sum of a_n,r over the services cached at t	the proposal's per-resource occupancy
imbalance, stranded W	U^max - min_r U_r and sum_r (U^max - U_r)	proposal eqs. (30)-(33)
violation rate	fraction of steps with max_r U_r,t > 1	goal G1 is about the TRUE constraints, not the reduced ones
Lambda_phi (eq. 18)	max_n phi(a_n) / max_r a_n,r	how much a slot reduction over-charges a service
surrogate regret	sum_t F_t(P_t) - min_P sum_t F_t(P), the minimum computed exactly by an LP	the comparator of Proposition 1, solved to optimality (not an approximation)
running time	1e6 * wall-clock time of the request loop / T	measures the algorithm only, not the catalogue or the trace
latency per unit resource	L / sum_t U^max_t	proposal eq. (34)

Result 1 - the adversarial family

the paper's Large Sequence, OPT = (k+2n)l





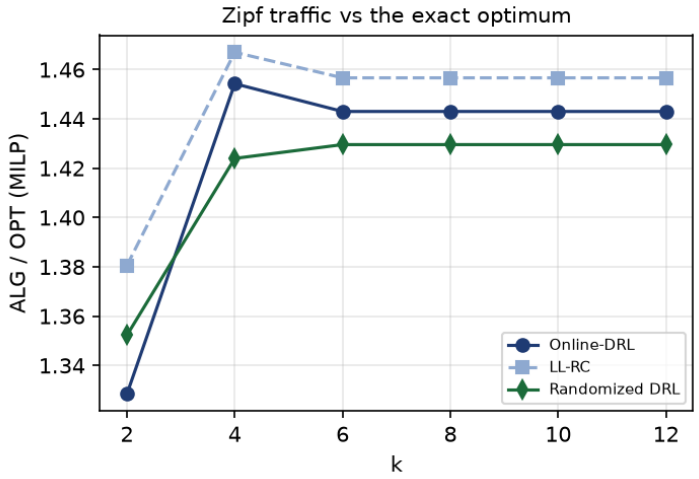
k	gamma det.	gamma R3	gamma rand.	cost det.	cost rand.
3	6.33	6.0	2.65	204	42
6	12.33	11.7	2.68	840	85
9	18.33	17.5	3.17	1,908	146
12	24.33	22.9	3.57	3,408	216
15	30.33	28.5	2.74	5,340	158
18	36.33	34.2	3.66	7,704	325
21	42.33	39.9	3.25	10,500	275
24	48.33	45.6	2.84	13,728	234

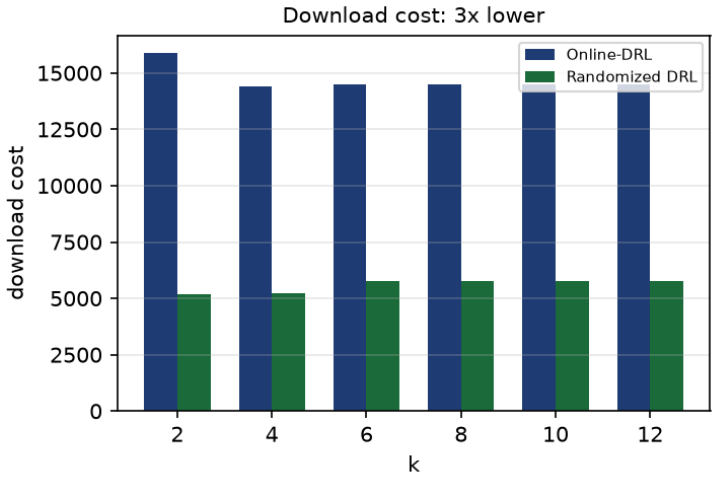
Takeaway:

Online-DRL is EXACTLY linear: gamma = 2.00k + 0.33 (R^2 = 1.000). R3 alone does not help (1.88k): the blow-up comes from the cache management, not from the download trigger. With R1+R2+R3 the ratio is FLAT (gamma ~ 3) and the download cost is 5-60x lower.

Result 2 - ordinary traffic: it EQUALIZES

Zipf/Poisson, offline optimum = MILP



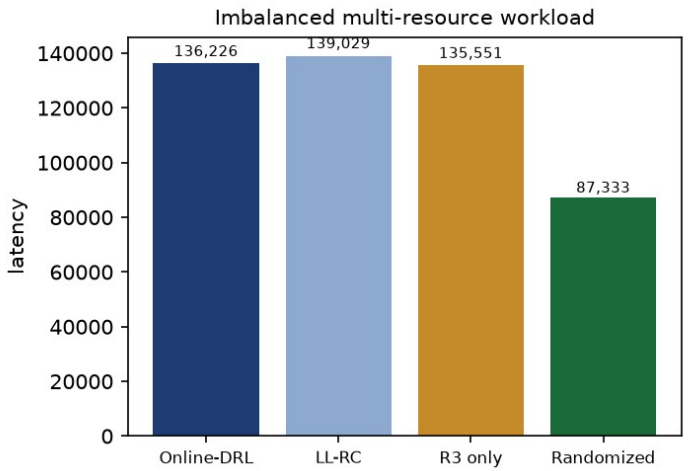


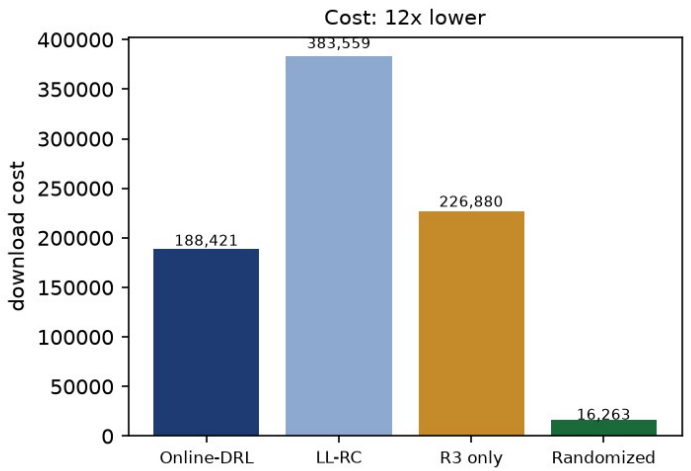
- All algorithms are flat in k: the resource limits bind before the cardinality does.
- Randomized 1.430 vs Online-DRL 1.443 - parity in the ratio, 3x less download cost.
- On SHORT traces (T = 120, 5 traces, MILP optimal) it is worse: 1.279 +- 0.138 vs 1.145 +- 0.072 (Online-DRL) and 1.102 (R3 only) - the fractional state has to be learned first.

Takeaway:
on easy workloads randomization buys COST and utilization, not ratio. Its advantage appears in the worst case.

Result 3 - the multi-resource setting

CPU, RAM and disk at the same time





algorithm	latency	cost	dl	U^max	imb	viol.
Online-DRL	136,226	188,421	517	0.786	0.459	0.00
LL-RC	139,029	383,559	1020	0.792	0.463	0.00
R3 only	135,551	226,880	611	0.787	0.461	0.00
Randomized DRL	87,333	16,263	40	0.740	0.334	0.00

- 5-13x fewer downloads, 30-36 % less latency, less imbalance and less stranded capacity, and 0 % violations of the three resources.
- Part of the gain is 'simply downloading less' on a stationary popularity - the specifically multi-resource advantage still needs a non-stationary resource mix.

The goals of the proposal - which ones hold

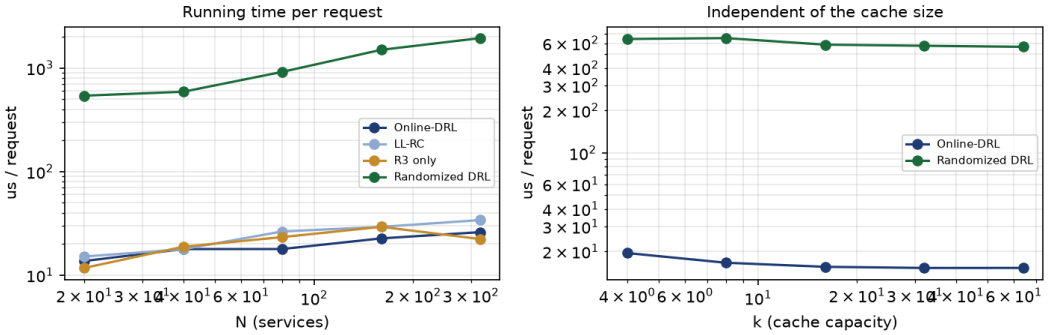
G1 - G5, measured

goal	measured	verdict
G1 feasibility	the un-repaired rounding violates the true limits in 40 % of the (path, step) pairs and reaches 1.73x the capacity; with the repair 0 %	CONFIRMED (repair mandatory)
G2 budget	respected at 20 %, 50 %, 100 % of the deterministic cost, for every seed; the budgeted deterministic algorithm loses 11.6 % latency	CONFIRMED
G3 regret (surrogate)	always below sqrt(N) l_max sqrt(T), exogenous and endogenous, with and without the peek update	CONFIRMED
G3 regret (literal)	3.7-9.6x the bound and growing with T	SURROGATE ONLY - see the next slides
G4 expected ratio	1.102 (R3), 1.145 (Online-DRL), 1.279 (Randomized) on short traces	CONFIRMED (R3 best on short traces)
G5 slot reductions	weights on the simplex or uniform shadow prices give Lambda < 1 and a lower latency but violate the true limits in 55-85 % of the steps	UNSAFE REDUCTIONS EXIST

Where Randomized Online-DRL LOSES

the honest list

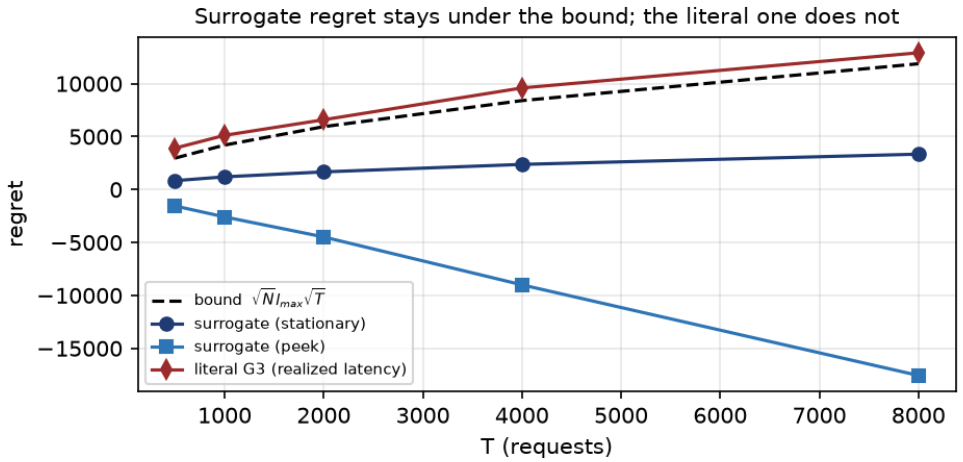
- Short traces. T = 120: 1.279 +- 0.138 vs 1.145 (Online-DRL) and 1.102 (R3 only). The fractional state is not learned yet.
- Computing time. 440-1 665 us per request against 13-26 us: every request does an O(N) projection, an O(K*N) rounding and an O(N) repair. About 35 ms of CPU per second at 1 000 requests/s.
- The literal form of goal G3. The bound only holds for the surrogate loss; the realized-latency version is violated by a factor 3.7-9.6.
- It never loses on resource feasibility: the repair step makes the constraints exact (0 % violations everywhere).



Terminology 1 - 'surrogate only'

goal G3 read carefully

- The regret we would LIKE to bound uses the realized latency: sum_t l - min_P sum_t l 1[n_t not in P].
- But that expression is not a function of p alone: whether a request is a hit, a delayed hit or a forward also depends on q_t (a download in flight) and dm_t (its remaining time), which are decided by the discrete download dynamics.
- The proposal therefore uses a SURROGATE loss that IS a function of p: F_t(P) = (1 - p_nt) * g_t, with g_t = (1-q_t) l + q_t min(l, dm_t) - 'a request that is not covered by the fractional cache costs g_t'.
- The bound sqrt(N) l_max sqrt(T) is the regret of the SURROGATE, because D_vec has diameter <= sqrt(N) and eta = sqrt(N)/(l_max sqrt(T)).
- Why the literal version fails: the fractional comparator may cache a service INSTANTLY, while the real system must first DOWNLOAD it - and the comparator is not charged for those downloads.
- Fixes: define G3 on the surrogate loss, charge the comparator for its downloads, or compare against a download-aware comparator.

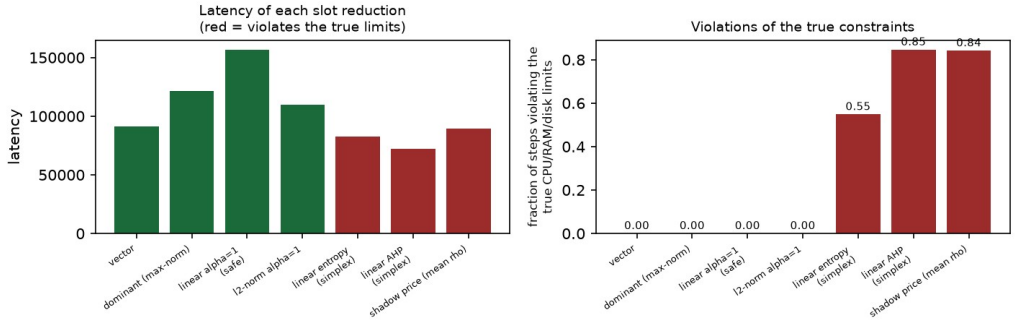


'Surrogate only' means:
the bound holds for the surrogate and only for the surrogate.

Terminology 2 - 'unsafe reduction'

goal G5: slot reductions

- A slot reduction replaces the three resource constraints by ONE scalar constraint sum_n phi(a_n) p_n <= 1, with a weight function phi (dominant resource, a linear or l_p mix, shadow prices, AHP or entropy weights).
- Lambda_phi = max_n phi(a_n)/max_r a_n,r (eq. 18) measures the damage: Lambda < 1 means the reduction charges a service LESS than its true bottleneck demand, i.e. the reduced constraint is WEAKER than the real one.
- Such a reduction is called UNSAFE: the cache it produces can exceed the real CPU/RAM/disk limits.
- The proposal's own warning: with weights on the simplex (sum_r alpha_r = 1) a disk-only service looks cheap; the reduction is safe only if alpha_r >= 1 for every r, at the price of wasting up to a factor R of the capacity.
- Measured: entropy, AHP and uniform shadow prices all have Lambda < 1 and a LOWER latency - and all three violate the true limits in 55-85 % of the steps (up to 2.37x the capacity). The safe reductions never violate, but waste capacity (linear alpha = 1: Lambda = 2.52, +71 % latency).



'Unsafe reduction' means:

a slot reduction is not a free simplification - the exact vector formulation dominates every safe reduction.

Summary

the verdict in one slide

regime	Online-DRL	Randomized DRL	verdict
adversarial patterns, large caches	gamma = 2k (linear)	gamma ~ 3 (flat)	randomized WINS
ordinary stationary traffic	1.443, 3x more cost	1.430, 3x less cost	randomized EQUALIZES and saves cost
multi-resource workload	136 226 latency	87 333 latency (-36 %)	randomized WINS
short traces	1.145	1.279	randomized LOSES (warm-up)
download budget	clipped, -11.6 % latency	respected in expectation and per seed	randomized WINS
computing time	13-26 us/req	440-1 665 us/req	randomized LOSES (25-70x)
goal G3	-	surrogate only	partial

Bottom line:
strictly better in the worst case, equal in the average case, slightly worse during the warm-up, and 25-70x more expensive per request. Still open: the real Google / SHARCNet traces, a non-stationary resource mix, and an adversarial family designed against the randomized algorithm (to separate O(log k) from O(k)).

How to reproduce

every number in this deck

- cd extracted/drl-sim && pip install -r requirements.txt && make test
- python experiments/run_all.py # ~30 min, writes results/*.json and results/*.png
- python docs/make_slides.py # this deck
- python docs/make_comparison_pdf.py # the 6-page comparison PDF
- python docs/make_contexts_pdf.py # the contexts PDF (surrogate, unsafe reduction)
- Documentation: docs/simulation_methodology.md (model, conditions, how every number is computed) and docs/results.md (all results).
- *Caveat: the Google / SHARCNet traces are NOT in this repository; the headline numbers here come from synthetic Zipf traffic and the paper's adversarial sequence.*